

QUANTUM PROCESSING CYCLE MODEL

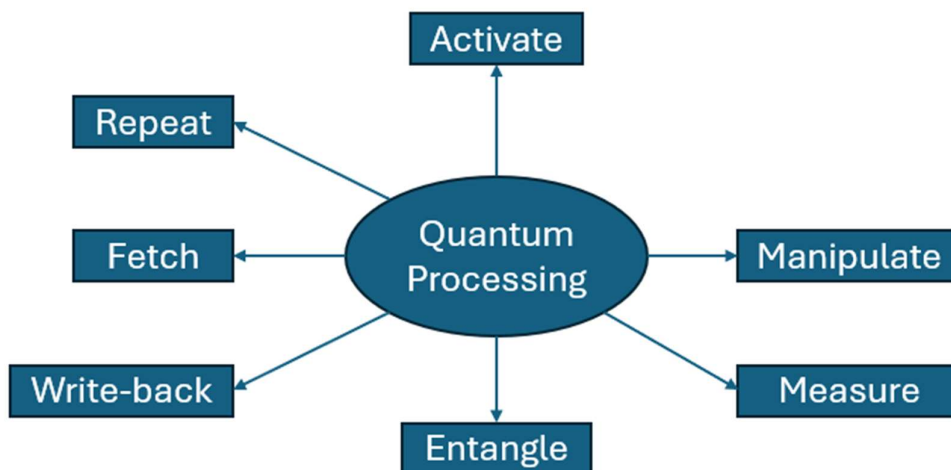
Contents

Introduction	2
Quantum Processing Cycle	3
Quantum Processing Cycle Trade-offs	4

Introduction

This Quantum Processing Cycle otherwise known as QPC was designed to make quantum processing and storage more efficient and reliable. This quantum processing cycle was created as quantum processing currently is inefficient and unreliable. It is important that quantum process is efficient as possible as quantum computing is extremely expensive. QPC does not change the fundamental processes of superposition, entanglement and measuring. This cycle focuses on making memory more efficient and making a simple way for a quantum processor to function.

This quantum processing cycle cannot promise perfect efficiency or performance at any point. However, this quantum processing cycle can promise to try improving efficiency and reliability of quantum processing. This cycle is made for systems that use quantum processing and the technical specifications within this document are for quantum engineers and experts. Make sure to adapt the details in this model as needed for implementation.



This spider diagram shows the main steps involved in this quantum processing cycle. For those who don't need what superposition is, it is when Qubits represent both 0 and 1 at the same time. Entanglement is when two or more particles become linked so that the state of one particle will affect the others. More details about the quantum processing cycle can be found in the section below.

Quantum Processing Cycle

- The first step is fetching.
- First, the quantum processor needs to find qubits in memory (RAM or other storage)
- Qubits are stored as stable 0s and 1s which is their collapsed state as it is more efficient and reliable compared to storing qubits in superposition.
- Second, the quantum processor must search through categories to find data more easily within memory.
- Third, the processor retrieves these qubits via a bus once it is found in memory.
- The second step is activating.
- After the quantum processor has fetched the qubits, they will be passed through a logic gate as they enter the processor.
- It is important that the gate isn't applied during transport as qubits decohere quickly.
- This logic gate is called Hadamard's gate which will convert the qubits from a collapsed state to superposition.
- This activation step only happens when the qubits are needed to ensure that qubits don't decohere in memory.
- The third step is entanglement.
- Once the qubits have been superposed, the quantum processor must entangle the qubits together.
- The quantum processor will use the CNOT gate otherwise known as the controlled NOT gate to entangle the superposed qubits.
- This occurs inside the processor as it makes sure that qubits cannot decohere as much compared to outside the processor.
- The fourth step is manipulating.
- After the quantum processor has successfully entangled to superposed qubits, it will apply multiple gates to execute the task needed.
- For example, Shor's algorithm is used to factor large numbers and the gates Hadamard, CNOT, phase gates and QFT are applied.
- The fifth step is measuring.
- After the task has been completed, the qubits are measured using a measurement operation.
- Measurement is done by looking at the collapsed state of the qubits which will give you the results you need.
- The sixth step is write back.
- Once the results have been found, they are written back to RAM and sorted into the needed category.
- The qubits are stored as 0s and 1s to ensure stable memory and to make sure the results can be used later for a similar or the same task.
- The seventh step is repeat which is where the cycle is repeated each time.

Quantum Processing Cycle Trade-offs

- QPC has trade-offs just like any other system I have created.
- There may also be further implementation challenges as quantum computing is still in early stages of development.
- I have identified 5 trade-offs.
- The first trade-off is time-consumption.
- This trade-off exists because it will take years to develop this cycle especially to make it work efficiently and reliable.
- This is because quantum computing is still in early development and combining it with classical computing will be difficult.
- This trade-off can be managed by working in teams, using pre-existing systems and structured development processes.
- The second trade-off is compatibility.
- This trade-off exists because this model demands a hybrid creation between classical and quantum computing systems.
- This is because a pure quantum computer would never be able to function efficiently without taking massive resources.
- This trade-off can be managed by testing current compatibility between both systems, planning out which components are classical vs quantum.
- The third trade-off is decoherence.
- This trade-off exists because superposed qubits will always decoherence within a quantum system.
- This is because physics limits how long a qubit can be superposed for as it isn't stable at all.
- This trade-off can be managed by storing qubits in stable memory, applying superposition at the processor and using superposed qubits quickly.
- The fourth trade-off is cost.
- This trade-off exists because a hybrid of quantum and classical computing systems will be extremely expensive.
- This is because quantum systems require extreme cooling systems which are expensive and current classical hardware is not cheap.
- This trade-off can be managed by allocating a budget to the project, using classical hardware as its cheaper and only implementing advanced cooling where needed.
- The fifth trade-off is complexity.
- This trade-off exists because combining category-based memory, classical computing systems and quantum systems can be extremely confusing.
- This is because category-based memory is relatively a new concept within hardware and hasn't been fully implemented yet.
- This trade-off can be managed by ensuring all systems are understood, allocating clear teams based on skills and structured project management.